STAT5003 ASM2 Unified Modelling Plan

STAT5003 Assignment 2 — Unified Modelling Plan & Team Allocation (v2)

Project: Predicting Match Outcomes in Europe's Top Five Football Leagues

Purpose: All five models use the same feature engineering, the same data split and the same evaluation metrics, so that the final model comparison is fair and feeds directly into both the presentation and the report.

> **Key change in v2.** The earlier plan used `position_diff` and built the baseline on `home_club_position` / `away_club_position`. These columns record **post-match** standings — verified in the A1 EDA, where on matchday 1 (when no club has a position yet) winners sat 2–7 and losers 13–17. They leak the target and have been removed. League position is now reconstructed from cumulative points in prior fixtures.

1. Task Definition

Item	Decision
Target	`result` : home_win / draw / away_win
Scope	Europe's top five leagues, 25,241 matches (2012/13–2025/26)
Task	Three-class classification using **pre-match information only**
Models	Multinomial Logistic Regression / LDA / kNN / SVM / Random Forest
Principle	One shared modelling dataset, one time split, one evaluation standard

2. Feature Engineering

All recent-form features use **only matches played before the fixture in question**. The current match and any future match must never enter the calculation.

Feature	Definition	Rationale
`table_pos_diff`	away pre-match position − home pre-match position	**Constructed**: ranked by cumulative points from earlier fixtures that season
`points_per_game_diff`	home season-to-date PPG − away season-to-date PPG	Overall season strength; more stable than position early in a season
`form_points_diff`	home last-5 PPG − away last-5 PPG	Recent form
`recent_goal_diff`	home last-5 goal difference per match − away equivalent	Recent attacking/defensive state
`venue_form_diff`	home recent home PPG − away recent away PPG	Home form vs away form
`h2h_diff`	home wins − away wins across the last **5** meetings	Head-to-head record
`rest_days_diff`	home rest days − away rest days	Fixture congestion
`competition_id`	League (categorical)	Controls for league context
`season_progress`	round ÷ maximum round **for that league, that season**	Early / mid / late season

Initial-condition rules (all five members must follow these)

Situation	Rule
No prior match that season (matchday 1)	All recent / form / table features = `NA` . **Do not use 0** — zero is a meaningful value and would be read by the model as "performing badly"

1–4 prior matches that season	Use **all available** matches, computed **per match** (see below)
Across seasons	Recent / form / table features use **the current season only**; no carry-over between seasons
`h2h_diff`	Uses **all historical meetings** in the dataset (across seasons), capped at the most recent 5; `NA` if no prior meeting
`rest_days_diff`	Uses each club's **previous fixture**, which may fall in the previous season; `NA` if none
Handling `NA`	Left entirely to the training-fold preprocessing in § 6. **Do not impute during feature engineering**

Per-match rates, not totals. All form features are computed as per-match averages rather than sums — for example, `form_points_diff` is home's points per match over the last 5 compared with away's. This keeps a club with 3 prior matches comparable to one with 5, so matchdays 2–5 need not be discarded.

Other implementation notes

- `season_progress`: `maximum_round` must be computed **per league, per season**. Ligue 1 reduced from 20 clubs to 18 in 2023/24, so its maximum round changed from 38 to 34; a global maximum would be wrong. `round` is a character field, so extract the digits first.

---

3. Variables That Must Not Be Used

Banned — known only after the match:

- `home_club_goals`, `away_club_goals`, `aggregate` — these are the target itself
- **`home_club_position`, `away_club_position`** — verified post-match standings; target leakage
- `attendance` — determined after kick-off and largely a function of stadium capacity
- `referee`, `home_club_formation`, `away_club_formation`, manager fields — unknown or unstable before kick-off

Excluded from the main models: the `clubs.csv` attributes (squad size, average age, foreigners percentage, national team players, stadium seats). These are a single scrape-time snapshot — verified identical across all 14 seasons for every club — so they cannot represent a team's state at the time of any given match. They may be discussed in the EDA and limitations.

---

4. Baseline and Refined Models

Stage	Features
Majority-class baseline	Always predict home_win (no variables at all)
Baseline model	`competition_id`, `season_progress` — season context only, no team-strength information
Refined model	`table_pos_diff`, `points_per_game_diff`, `form_points_diff`, `recent_goal_diff`, `venue_form_diff`, `h2h_diff`, `rest_days_diff`, `competition_id`, `season_progress`

Reference point: always predicting home_win gives an accuracy of **44.3%**. Every model must be compared against this.

---

5. Data Split (fixed — not open for discussion)

- Sort all matches by `date`, earliest to latest.
- **Training:** `season <= 2022` (seasons 2012/13 – 2022/23)
- **Final Test:** `season >= 2023` (seasons 2023/24 – 2025/26)
- In R: `train <- data[season <= 2022]`, `test <- data[season >= 2023]`
- This is approximately an 80/20 split. **No random splitting** — home advantage shifted markedly around 2020, so a random split would let models learn from a regime unavailable at prediction time.
- All five members use this split. Nobody adjusts it independently.
- Within training, use **5-fold expanding-window (rolling-origin) validation** for tuning and model selection.
- The Final Test set is used **once**, after the final model is chosen.

---

6. Preprocessing Rules

- Missing-value imputation, standardisation and feature selection are **estimated on the training fold only**, then applied to validation and test.
- kNN and SVM require numeric variables to be standardised. MLR, LDA and RF are handled according to each method's own requirements.
- Categorical variables use a single, reproducible encoding scheme.
- No preprocessing step may use future data or any information from the final test set.

---

7. Evaluation Metrics

- **Primary: Macro-F1** — determines the final model choice
- Supporting: Accuracy, Balanced Accuracy
- Per-class: Precision, Recall and F1 for Home Win / Draw / Away Win
- Diagnostic: Confusion Matrix
- **Watch draw recall.** Draws are the minority class at 25.1%, and models may under-predict them. Where that happens, Macro-F1 reveals the problem more clearly than Accuracy — which is precisely why it is the primary metric.

---

8. Model Allocation

Member	Model	Main tasks
Member 1	Multinomial Logistic Regression	Baseline + refined; handle collinearity and feature selection; report standard metrics
Member 2	LDA	Baseline + refined; check highly correlated variables and model assumptions; report standard metrics
Member 3	kNN	Standardisation; tune *k*; report standard metrics
Member 4	SVM	Tune kernel / cost / gamma; **time a fit on a 2,000-row subsample before choosing grid size**; fall back to a linear kernel if needed
Member 5	Random Forest	Tune mtry / node size; summarise feature importance; **also owns the split and the shared scoring function**

Blocking deliverables (everyone else waits on these)

Task	Owner	Deliverable
Publish final column names	Member 2	Exact column names for every §2 feature — **published before the dataset is built**, so the other four can write their model code while waiting
Feature engineering	Member 2	Modelling dataset containing all §2 features, saved as `.rds`
Shared scoring function	Member 5	`evaluate_model(truth, pred)` returning Accuracy, Balanced Accuracy, Macro-F1, per-class Precision/Recall/F1, and the Confusion Matrix
Fold structure	Member 5	5-fold expanding-window indices over the training set, saved as a loadable object

All five members must `source()` Member 5's scoring function. Nobody implements their own Macro-F1 — otherwise the comparison table is meaningless.

---

9. Standard Reporting Format (every member submits this)

- Model name
- Features used
- Preprocessing applied
- Baseline result
- Best hyperparameters / refinement
- Validation Macro-F1
- Test Accuracy
- Test Balanced Accuracy
- Test Macro-F1
- Precision, Recall and F1 for Home / Draw / Away
- Confusion Matrix
- 2–3 sentences on the main finding
- 1 sentence on a limitation

---

10. Final Comparison, Presentation and Report

Once all five models are complete, compare them on the same Final Test set. Rank primarily by **Macro-F1**, and use Accuracy, Balanced Accuracy and the confusion matrices to explain *why* the models differ.

Presentation (non-specialist audience, 7 minutes): problem → data → feature engineering → preprocessing → model refinement → model comparison → best model → final insight. Avoid technical terms such as kernels, covariance matrices and one-versus-one voting.

Report (10-page limit): full methodology, tuning process, unified evaluation, model limitations, and a direct answer to the research question.

Interpreting results. Match outcomes carry substantial inherent randomness. Model performance only needs to be judged against the two baselines in § 4. If the refined model clearly beats the 44.3% majority-class baseline, the feature engineering has done its job. Even a modest absolute accuracy is a meaningful finding — the conclusion is that pre-match information explains only part of the outcome. **Do not cite unverified benchmark figures from the literature in the report.**

Execution Order

Raw data leakage check → feature engineering → chronological split → fold-wise preprocessing → model fitting & tuning → final test evaluation → model comparison → presentation / report